Spec-Driven Development (SDD)

SDD Overview

- Disclaimers
- Engineering with AI
- Foundational concepts
- Context engineering
- SDD in practice

Disclaimers

Disclaimer - all is change

AI engineering practices are evolving rapidly. In cases like MCP, something that was completely new becomes industry standard within a year. Engineers are looking for and building out abstractions for common problems and adopting useful frameworks and patterns quickly. The dust has not settled, **things will continue to change, standards are still emerging**.

Disclaimer - it's not free

Use of AI coding agents is **required only to a limited extent for this class** (achievable with free tooling), but many of us are using these tools anyway. What we are learning in this class is a set of general processes around software development with AI. The processes should be applicable to whatever AI coding tools you use.

There are free, limited tiers available to you, but **it may be worth investing in a subscription to go deeper with your learning**.

Disclaimer - keep it educational

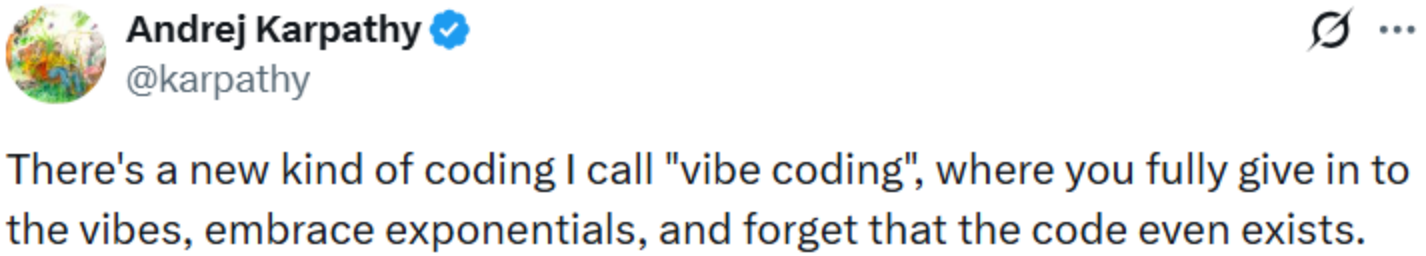
Using agentic tooling is a critical skill for developers to learn but **it should not overshadow your other educational goals**. You must use discretion and still work in a manual way when needed to develop your baseline skills.

Using agents is not an excuse to not understanding your code or codebase. The standard (for this class and assessment) is still high:

You must understand and be able to explain every line of code in your codebase. You are still responsible for the code, standards and architectural decisions being made.

Engineering with AI

Vibe coding



Andrej Karpathy (founding member of OpenAI) coined the term "vibe coding".

We have all had some experience with this. What do we think? Is it a suitable approach for software development? Why or why not? In what cases is it most appropriate?

Vibe coding problems

In general, vibe coding or even simple prompt-based feature engineering fails beyond simple prototypes and small applications for a variety of reasons including:

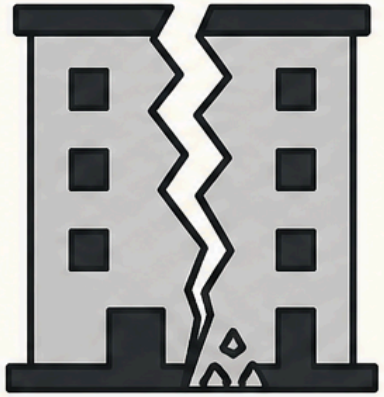
- **Architectural drift** - Each prompt solves a local task but not necessarily in an architecturally consistent way. The system accumulates multiple and inconsistent implementations and architectural choices. Inconsistent schemas, diverging error handling, duplicated auth. These inconsistencies can compound exponentially over time as the inconsistencies pollute the agent's inferred context.
- **Missing context** - Conventions live outside the code (validate at service layer, soft deletes, localized errors). These are invisible to an AI given only the task leading to incorrect or missing implementations due to missing details.
- **The plausibility gap** - AI code looks production-quality (formatting, names, patterns) leading to quick approval of large changes with potential bugs.

THREE FAILURE MODES



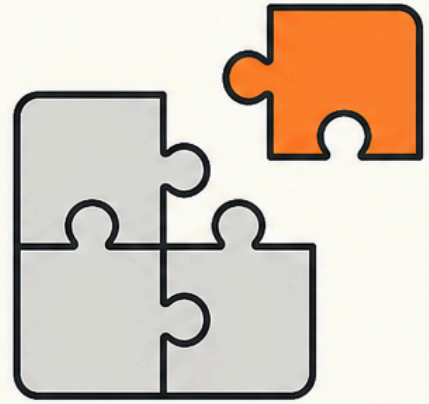
PLAUSIBILITY GAP

Looks like production code



ARCHITECTURAL DRIFT

Each prompt ignores the whole



MISSING CONTEXT

Conventions the model cannot see

SDD as a solution to vibe coding limitations

Software engineering has known for decades that **ambiguous requirements are expensive**. Boehm (1981): a defect found in production can cost far more than one caught in design.

What changed in 2025 is speed. With **vibe coding** (describe the vibe, accept the diffs, ship), **prototyping got cheaper but production defects got cheaper to create at volume**.

Spec-driven development *is the emerging standard around developing applications in a rigorous, detailed and incremental way using text-based specification documents to guide the agent during development.*

TWO WAYS TO BUILD

VIBE CODING



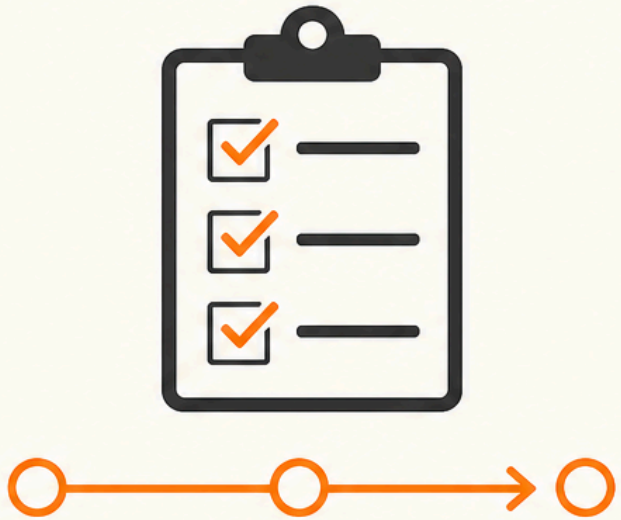
Vague prompt

Looks right

Breaks in production



SPEC-DRIVEN



Clear intent

Explicit constraints

Verify before merge



Why the old workflow no longer scales

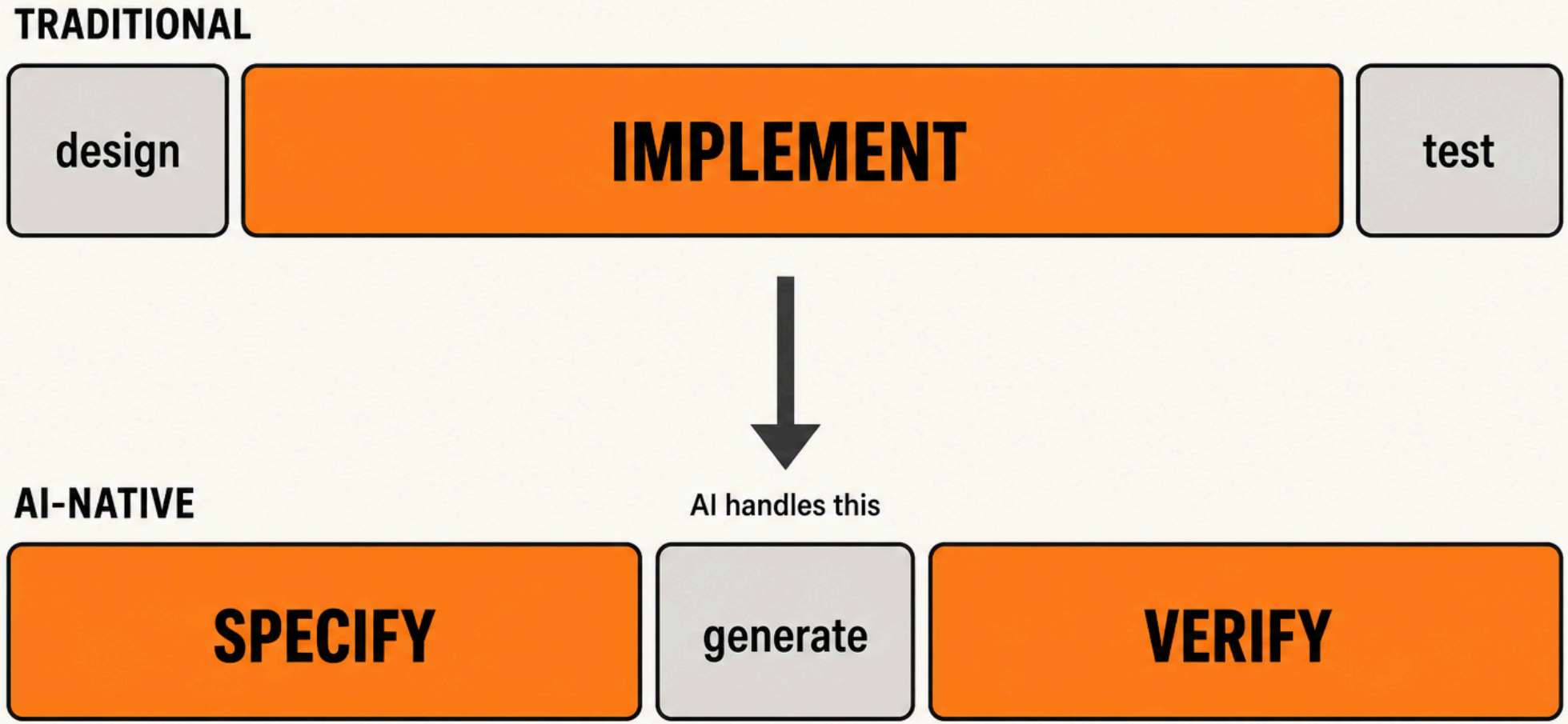
Traditional development assumed **implementation speed was the bottleneck**, and that you were the author of every line. Both assumptions no longer hold.

You used to discover the design *by writing the code*. Edge cases showed up in tests. The data model changed when you hit the third entity. That worked because ***you*** **accumulated context and understanding as you went**.

An agent does not. It starts fresh at each context boundary. Underspecify, and it invents assumptions - then builds the next prompt on top of them.

So the expensive work moves upstream (clear spec) and downstream (rigorous verification). Generation in the middle is the cheap part.

WHERE THE BOTTLENECK MOVED



You are no longer paid mainly to type the code

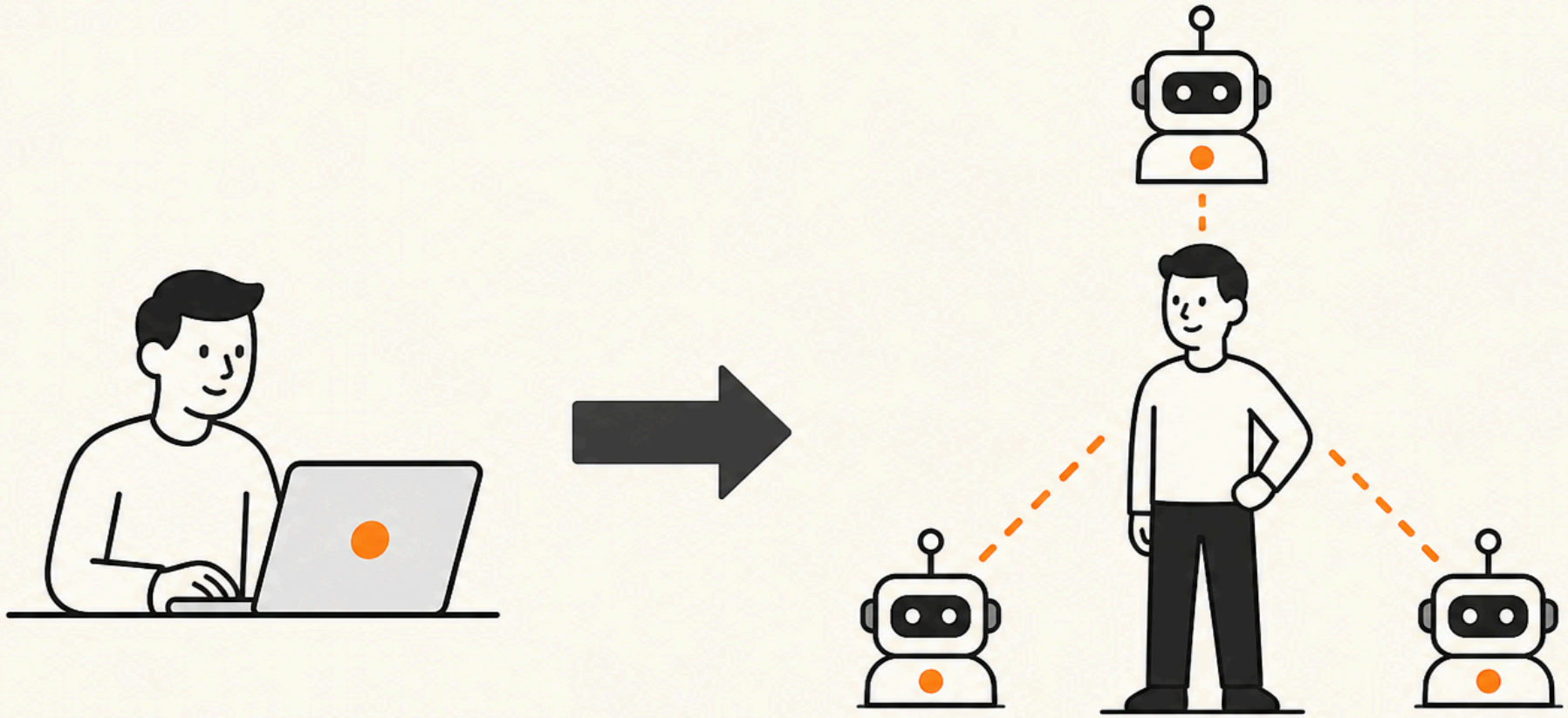
When an agent can draft a CRUD endpoint in 30 seconds, the work **moves toward the spec**:

- What should this actually do?
- What should not change?
- What happens on bad input or under load?
- Is this the right design for *this* system?

...and **the verification**

- Is there a verifiable way of proving the work has been completed?
- Are tests comprehensive and passing?

FROM IMPLEMENTER TO ORCHESTRATOR

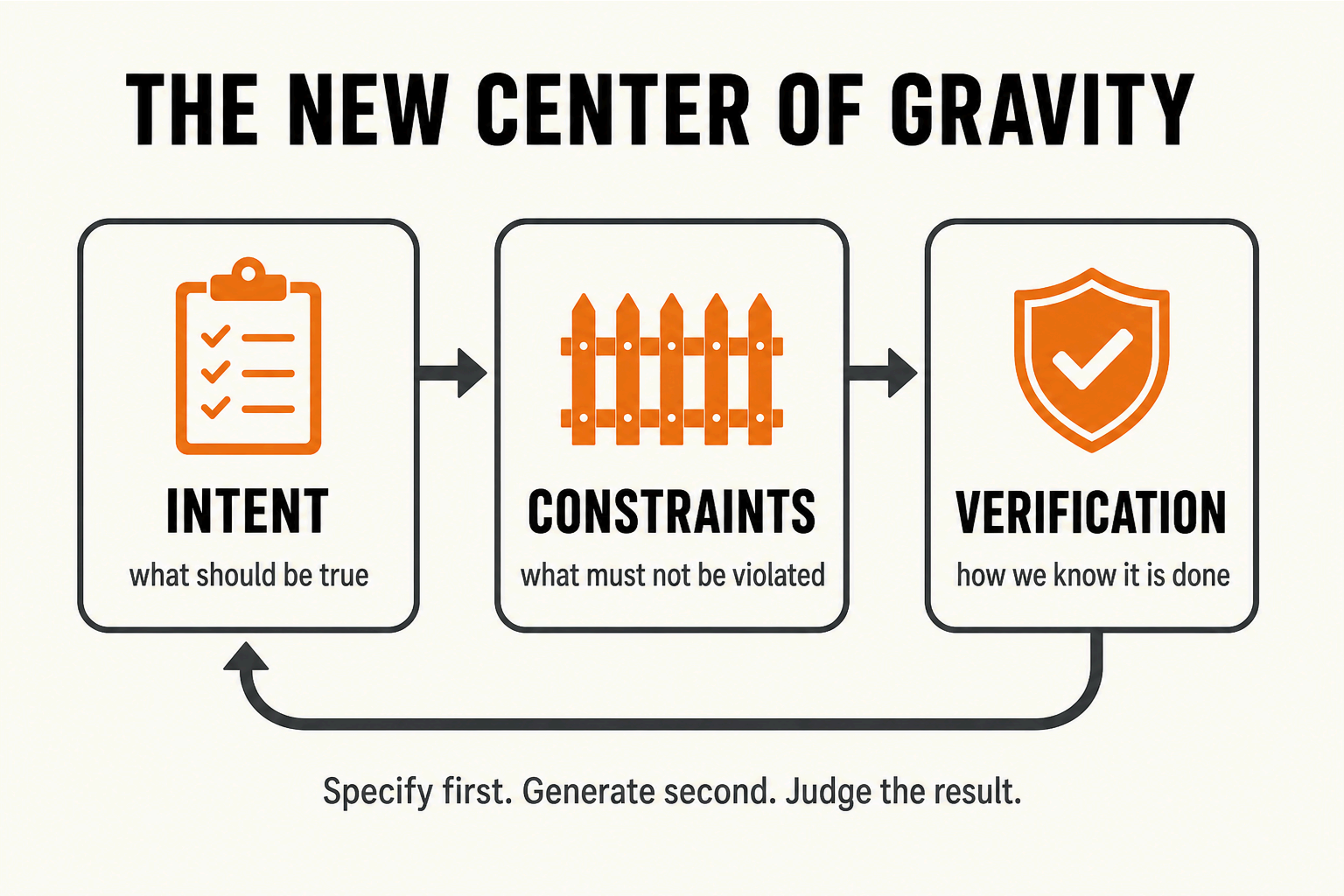


IMPLEMENTER

ORCHESTRATOR

SDD development cycle

SDD relies on a cycle where intent and constraints are defined, implementation is completed, and verification is performed.



Intent: what should be true when you are done

Intent is an **outcome**, not a list of implementation steps.

Weak: "Add a user profile page."

Strong: "Return display name, avatar URL, member-since date, and the 2 most recent public posts. 404 if the user does not exist. Hide email from unauthenticated callers. Cache 60s using the existing user-resource cache key convention."

Vague intent produces vague code. The model will fill every gap with **training-data defaults** for "what a profile page usually looks like." Those defaults will match your system only by luck.

Constraints: the walls the agent cannot see

The most common AI failure: the agent **violates a constraint it was never told**.

The code passes tests, merges, and the violation shows up later - often when it is expensive.

Write them down **before** the task:

- **Technical** - existing auth library, p99 under 200ms, no new database
- **Business** - all locales, backward-compatible with v2, no emails to anonymous users
- **Quality** - coverage bar, security scan, project naming conventions

If it is not in the spec, the model is free to invent something plausible.

Verification: your judgment replaces the model's confidence

AI is confident. It can also be wrong: hallucinated APIs, misread code, locally smart assumptions that are globally false.

Verification layers including:

1. **Automated** - unit/integration tests, linters, types, CI. Catches the embarrassing class of errors quickly.
2. **Structured review** - working but problematic implentations, poor conventions, shortcuts that will hurt later?
3. **Acceptance** - run it the way a real user would.

Tests the agent wrote are necessary and **not sufficient**. Agents can generate tests that pass without covering the behavior that matters.

Review against **criteria**, not against a vibe that it "looks right."

HUMAN IN THE LOOP



Context engineering

Skillfully filling the context window

> “Context engineering is the delicate art and science of filling the context window with just the right information for the next step.” - Andrej Karpathy

Everything the model knows about your task, codebase, standards, and intent must arrive in the **context window**. It cannot browse, remember last Tuesday, or look up your testing library unless something delivers that information.

You control what it receives.

System prompt, AGENTS.md, rules files, referenced snippets, tool definitions, conversation history - the totality of what the model sees is what it begins reasoning with.

ATTENTION IS A GRADIENT



Reliable at the edges. Weak in the middle.

ADVERTISED:
200k tokens

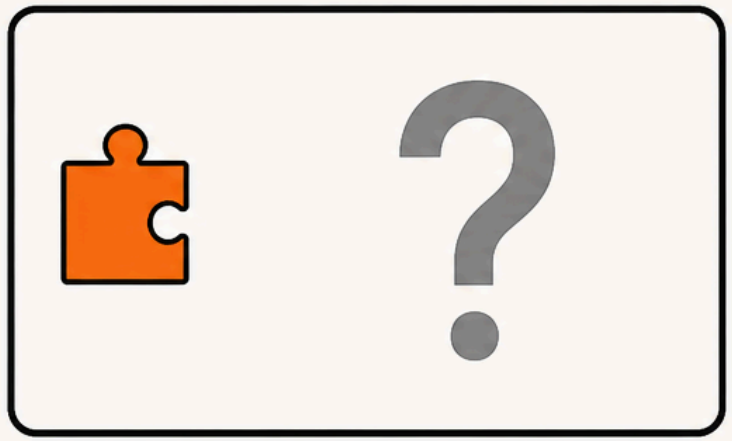
EFFECTIVE:
~100k tokens

Plan sessions around half the advertised window.

Pairwise attention scales n^2. More tokens, thinner attention.

TWO CONTEXT FAILURES

TOO LITTLE



GAPS FILLED FROM TRAINING DATA

FINE IN THE ABSTRACT

WRONG FOR YOUR SYSTEM

TOO MUCH



WORSE OUTPUT, SLOWER, COSTLIER

ATTENTION STRETCHED THIN

MORE IS NOT MORE FOCUS

ENOUGH TO REASON WELL. STRUCTURED SO ATTENTION LANDS WHERE IT SHOULD.

AGENTS.md

AGENTS.md

A README for agents

README.md is for a person.

AGENTS.md is instructions and context for a coding agent.

Keep the two files separate. Most coding agents read AGENTS.md.

AGENTS.md can be thought of as the **agent's longterm project memory**.

A README FOR AGENTS



README.md

for people

What the project does
How to install it
How to contribute

PEOPLE



AGENTS.md

for coding agents

Setup commands
Code style
Testing

AGENTS

Same repo. Different reader.

Example AGENTS.md

The headings below are a **choice**, not a template.

```
# AGENTS.md

## Setup

- Install dependencies: `composer install`
- Run tests: `vendor/bin/phpunit`

## Code style

- PHP 8.3 with `declare(strict_types=1);`
- PSR-12, one class per file

## Testing

- Add or update a PHPUnit test when behavior changes
- Run `vendor/bin/phpunit` and fix failures before finishing
```

WRITE THE DECISION DOWN



The next agent starts from the decision.

Sources

Core documentation:

- agents.md
- agentsmd/agents.md

Skills

A workflow you invoke

A skill is a folder with a `SKILL.md` file: instructions for one workflow. Run it by naming it or with an @-mention.

Skills can also contain **reference material** (official documentation, standards, etc.) **and tools or scripts** (usually JS or Python) that can be used as part of the skill (to format, parse, request data, etc.).

Name, description, and an explicit call

The file opens with YAML frontmatter.

- `name` — the skill's name
- `description` — when to use it
- `disable-model-invocation: true` — do not start this skill unless you invoke it

The four skills in the source repo set that flag.

Example skill

```
---
name: add-endpoint
description: Adds one REST endpoint using this repo's router, validation, and tests. Use when the user asks to add an API endpoint.
disable-model-invocation: true
---

# Add an endpoint

1. Add the route in the existing router.
2. Validate input with the project's validator.
3. Add or update a test.
4. Run `npm test` and fix failures before finishing.
```

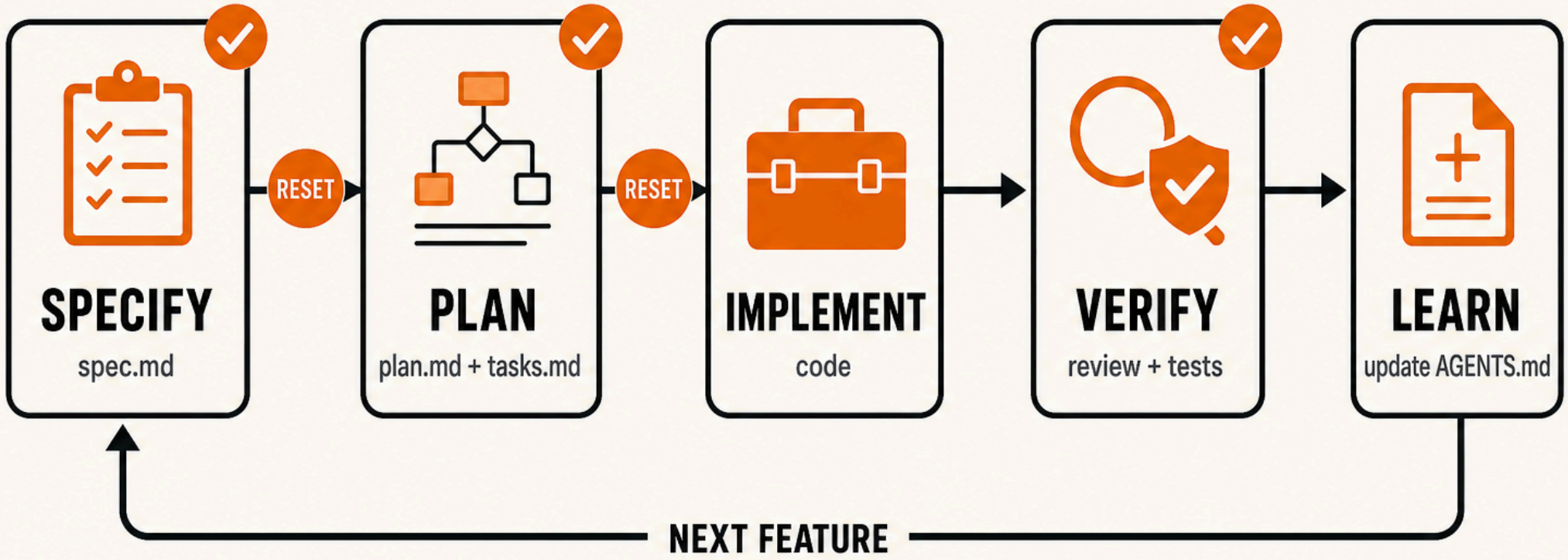
Four skills in the source repo

Run them in this order.

- **spec-init** — writes `specs/NNN-slug/spec.md` from its template, using only what you already said. Does not grill, plan, or implement.
- **spec-grill-me** — asks short rounds of questions, then rewrites that spec in place (Goal, Requirements, Acceptance Criteria, Out of Scope). Does not plan or implement.
- **spec-plan** — writes `plan.md` and `task.md` next to the spec. Does not implement or edit the spec.
- **spec-execute** — carries out unchecked tasks and checks them off. Does not rewrite the spec, plan, or tasks, and does not commit.

SDD in practice

THE SDD PRACTICE LOOP



Clear context between stages so leftover conversation does not pollute the next one.

Defining a spec

A spec is just a markdown file, committed to your repo, outlining details for a specific feature.

A spec should clearly document your *feature goal, requirements, acceptance criteria and scope*.

The structure of the file and the organization of the spec directory may vary somewhat according to your team's conventions and/or the framework you are using.

Example spec

```
## Goal

Add in-memory caching for the /api/products endpoint to reduce database load during high traffic

## Requirements

- Use the existing in-process cache; no new infrastructure
- Cache TTL: 5 minutes
- Cache key: product category ID (not the full query string)
- Cache must be invalidated on product updates via the existing product.updated event

## Acceptance Criteria

- First request for a category returns data from the database
- Subsequent requests within the TTL return data from the cache (verifiable via logging)
- A product.updated event causes the relevant category's cache to be invalidated within 1 second
- Cache hit/miss ratio is visible in existing monitoring dashboards

## Out of Scope

- Distributed caching across multiple instances
- Caching for other endpoints
- Cache warming on startup
```

Spec iteration, grill me

A common strategy for spec improvement and iteration is through the process of *grilling*. *Grilling reverses the roles.*

While grilling, the agent examines your spec file, identifies areas of uncertainty and undefined implementation choices, and asks you, interactively, to improve clarity.

Afterwards, the spec is updated with additional details the agent should need to begin implementation planning.



Planning

After the spec is defined, we will prompt the agent to develop two additional documents before implementation. **A plan document and a task document.**

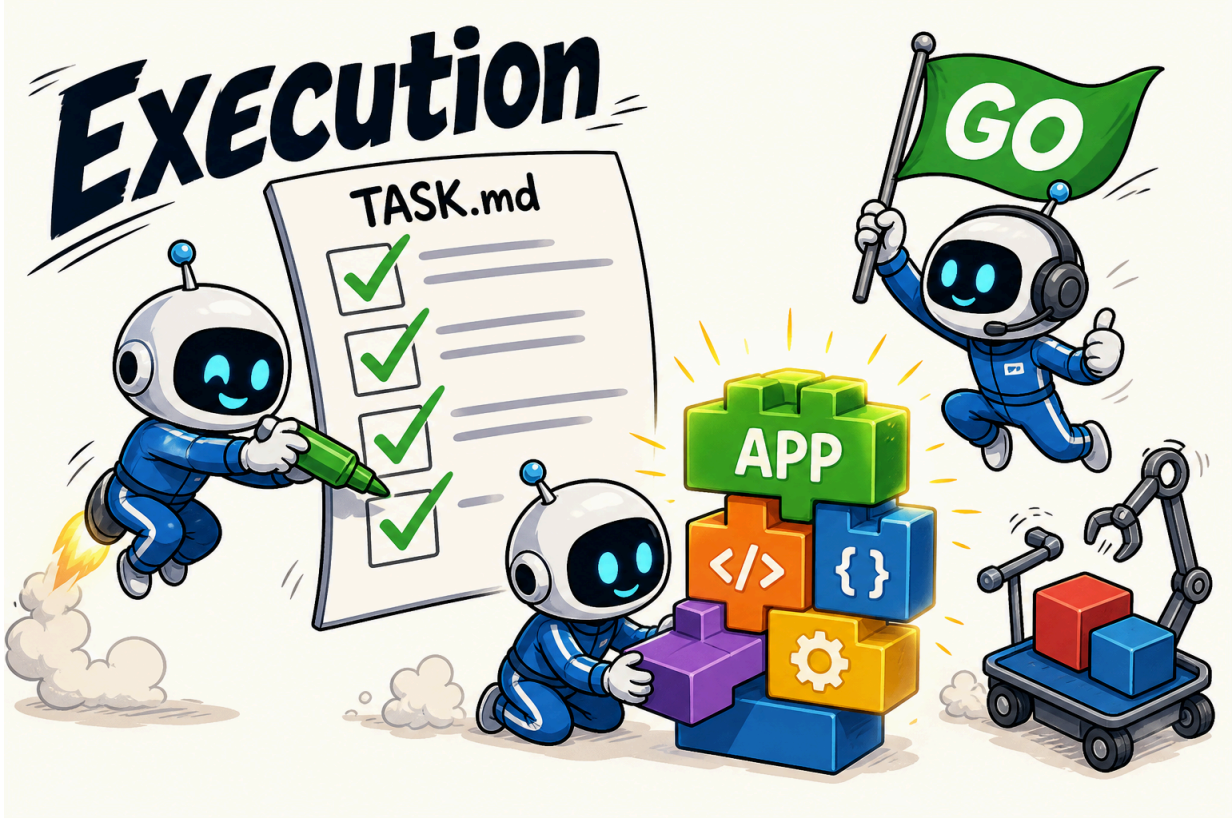
We will use the **spec-plan** skill to create the plan and task documents. During planning, the agent is instructed to review the spec document and create a concrete plan: which files to modify, which files provide needed context or must be interacted with, example code snippets, and other implementation details.

After completing the plan, the agent will create a final task.md document. This document includes the plan as context and a grouped checklist of items that must be completed.

Execution!

After planning has been completed and verified, you should clear context and have the agent implement your task list.

Implement the task list with the **spec-execute** skill.



Verification

As noted earlier, verification is an essential final part of the SDD cycle.

You should review the diff, especially closely for high-stakes updates, and apply a layered testing approach (user, e2e, unit).

Tests should be included as part of your rules and added for each new feature the agent develops.

Architectural documentation

Finally, the agent should be instructed as part of the general rules to document in the rules or AGENTS.md file any architectural decisions or standards that were determined during the SDD cycle so these standards are part of future agents' working context.

Let's do it!

Let's take a look at a sample code repo and develop a new feature using SDD.

